

AI API Validation Tool

Solution Architecture and Delivery Structure

This document describes an automated QA assistant that fetches Jira requirements, generates API test scenarios from ticket descriptions and acceptance criteria, mutates request payloads, executes APIs, validates responses, and produces QA-verifiable reports.

Architecture Overview

The platform should combine Jira requirements with deterministic API knowledge. Jira gives intent, but reliable automation also needs OpenAPI or Swagger contracts, Postman collections, sample payloads, environment configuration, authentication rules, reusable test data, and validation logic.

```
Jira Ticket -> Jira Integration Service -> Requirement Parser -> AI Test Scenario Generator
OpenAPI / Swagger / Postman -> API Contract Repository -> Scenario Generator
Scenario Generator + Test Data Repository -> Payload Builder -> API Execution Engine
Environment + Auth Manager -> API Execution Engine -> Response Validator
Response Validator -> AI Result Analyzer -> QA Review Report -> Dashboard / Jira
```

Core Components

1. Jira Integration Service

Fetches Jira tickets, descriptions, acceptance criteria, labels, comments, attachments, and linked stories. It should also publish generated reports back to Jira as comments or attachments.

- Authenticate with Jira using OAuth or API token.
- Fetch tickets by ticket key, JQL, sprint, status, assignee, or webhook trigger.
- Extract summary, description, acceptance criteria, business rules, attachments, and linked issues.
- Push the final QA report back to Jira for traceability.

2. Requirement Parser

Converts unstructured Jira text into structured requirements that can be mapped to APIs and test scenarios.

- Identify affected feature, entities, endpoints, user actions, preconditions, and business rules.
- Classify positive flows, negative flows, boundary conditions, auth checks, and missing details.
- Produce a structured intermediate model that the test generator can use safely.

```
{
  "feature": "Refund payment",
  "rules": [
    "Refund allowed only for settled payments",
    "Refund amount must be less than or equal to captured amount"
  ],
  "entities": ["payment", "refund"],
  "missingInfo": ["Exact refund endpoint not mentioned"]
}
```

3. API Contract Repository

Stores the API truth source. This component helps the AI map Jira requirements to actual endpoints, request schemas, response schemas, auth requirements, and expected status codes.

- Import OpenAPI or Swagger files.
- Import Postman collections and sample request/response payloads.
- Index internal API docs, historical tests, and known defect patterns.

- Version contracts so generated tests can be traced to a specific API definition.

4. AI Test Scenario Generator

Generates reviewable test scenarios using Jira requirements, API contracts, historical test patterns, and business rules.

- Generate positive, negative, boundary, required-field, data-type, auth, idempotency, and state-transition scenarios.
- Trace each scenario back to a Jira acceptance criterion or business rule.
- Return structured JSON, not free-form text, so scenarios can be edited, stored, and executed.

```
{
  "scenarioId": "PAY-1234-TC-004",
  "title": "Refund amount greater than captured amount",
  "type": "negative",
  "endpoint": "POST /payments/{paymentId}/refunds",
  "payloadMutation": { "amount": 150, "reason": "Customer request" },
  "expectedStatus": 400
}
```

5. Payload Builder and Mutation Engine

Builds executable requests from base payloads and modifies them according to each generated test case.

- Support replace, remove, nullify, empty string, invalid type, max length, boundary min/max, duplicate request, and special-character mutations.
- Resolve dynamic values from previous API responses, fixtures, test data, or environment variables.
- Keep original and mutated payloads in the report so QA can inspect exactly what was sent.

6. Test Data Manager

Prepares valid system state before API execution and cleans it up after test runs where possible.

- Create prerequisite users, orders, payments, tokens, accounts, and other domain records.
- Fetch valid IDs from APIs or allowed databases.
- Mask sensitive data and avoid production data by default.
- Maintain reusable fixtures for common preconditions such as settled payment or inactive user.

7. API Execution Engine

Runs generated scenarios against the selected environment and captures complete evidence for review.

- Execute REST calls with generated headers, path params, query params, and payloads.
- Support chained requests, token refresh, retries for transient failures, and async job polling.
- Capture request, response, status code, headers, latency, timestamps, and correlation IDs.
- Run jobs asynchronously for large ticket batches.

8. Response Validator

Uses deterministic checks wherever possible. The AI can help explain results, but pass/fail decisions should rely on explicit assertions.

- Validate HTTP status code, response schema, required fields, error codes, and field values.
- Validate response time SLA and contract compliance.
- Optionally validate downstream state using read APIs or approved database checks.
- Classify results as passed, failed, blocked, skipped, or needs manual review.

9. AI Result Analyzer

Summarizes execution results in QA-friendly language and explains likely causes of failures.

- Explain which acceptance criterion appears to be violated.

- Classify failures as functional, contract, environment, data, auth, or flaky execution issues.
- Recommend specific manual QA verification steps.
- Generate defect draft text without automatically creating defects in the MVP.

10. QA Review Dashboard

Provides a human review layer so QA can approve, reject, edit, rerun, or export generated scenarios and results.

- Ticket-wise run summary, generated scenarios, request/response evidence, pass/fail trend, and AI reasoning.
- Manual QA verdict with comments.
- Exportable PDF/HTML/JSON report and Jira publishing action.
- Optional integrations with Xray, Zephyr, TestRail, Slack, Teams, or CI/CD.

End-to-End Flow

1. QA selects a Jira ticket, sprint, or JQL query.
2. The tool fetches ticket details and linked context from Jira.
3. The requirement parser extracts structured business rules and missing information.
4. The API mapper finds relevant endpoints from OpenAPI, Swagger, Postman, or indexed API docs.
5. The AI generator creates structured and traceable test scenarios.
6. QA optionally reviews and approves generated scenarios before execution.
7. The payload engine creates valid and mutated request bodies for each scenario.
8. The execution engine runs APIs in the selected non-production environment.
9. Validators check status, schema, body, business assertions, and system state.
10. The result analyzer summarizes failures and recommends QA follow-up.
11. The dashboard shows the final report and can publish it back to Jira.

Recommended Technology Stack

Backend	Python with FastAPI, Celery or RQ workers, PostgreSQL, Redis, httpx, Pydantic, jsonschema, and OpenAPI parsing libraries.
AI Layer	LLM-based requirement parsing, scenario generation, result explanation, structured JSON outputs, RAG over API docs and historical tests, and prompt/version governance.
Knowledge Store	PostgreSQL plus pgvector or another vector database for API docs, sample payloads, historical scenarios, and defect patterns.
Frontend	React or Next.js dashboard for ticket selection, scenario review, execution monitoring, evidence inspection, and report export.
Integrations	Jira REST API, OpenAPI/Swagger, Postman collections, CI/CD, Slack or Teams, and optional test management tools such as Xray, Zephyr, or TestRail.

Suggested Project Structure

```

ai-api-validation-tool/
backend/
app/
integrations/ # Jira, Postman, OpenAPI, test-management clients
requirements/ # Requirement parser, models, prompt templates
knowledge_base/ # Indexing, retrieval, embeddings
test_generation/ # Scenario generation and coverage classification
payload_engine/ # Base payloads, mutations, generated data
execution/ # Runner, HTTP client, auth, environments, chains
validation/ # Status, schema, assertion, business validators
analysis/ # Result analyzer, defect suggester, QA summary
reports/ # PDF/HTML/JSON reports and Jira publishing
db/ # Models, migrations, repositories
workers/ # Async job definitions
config/ # Settings and environment configs
tests/
unit/
integration/
contract/
frontend/
src/
views/
TicketView.tsx
ScenarioReview.tsx
ExecutionReport.tsx
docs/
openapi-specs/
postman-collections/
sample-payloads/
docker-compose.yml
README.md

```

MVP Scope

- Fetch Jira ticket details and acceptance criteria.
- Import OpenAPI or Postman definitions.
- Generate structured AI test scenarios with traceability to Jira AC.
- Review scenarios before execution.
- Mutate payloads for positive, negative, and boundary cases.
- Execute APIs against a selected non-production environment.
- Validate status code, schema, required fields, and configured assertions.
- Generate PDF/HTML/JSON reports and publish summary back to Jira.

Do not start with fully autonomous defect creation. In the MVP, generate a defect draft and let QA review it before creating or linking bugs.

Important Guardrails

- Do not execute generated tests against production by default.
- Require environment-level permissions and explicit execution approval.
- Use deterministic validators for pass/fail decisions wherever possible.
- Keep generated tests editable and auditable before execution.
- Store request/response evidence and mask secrets, tokens, PII, and payment data.
- Version prompts, contracts, scenarios, and execution results.
- Track which Jira acceptance criterion produced each test case.
- Classify uncertain results as needs manual review instead of forcing pass/fail.

Example QA Output

Ticket: PAY-1234
Generated Scenarios: 18
Executed: 18
Passed: 14
Failed: 3
Blocked: 1

Failures:

1. Refund amount greater than captured amount returned 201 instead of 400.
2. Missing reason field returned 500 instead of 400.
3. Duplicate refund request created two refunds instead of one.

QA Recommendation:

Manual verification required for failed scenarios. Failures appear related to refund validation and idempotency handling.

Delivery Roadmap

Phase 1 - Foundation

- Build Jira fetch, OpenAPI/Postman import, environment config, and basic scenario generation.
- Create report storage and a simple execution history model.

Phase 2 - Execution

- Implement payload mutation, API execution, auth management, and deterministic validation.
- Add request/response evidence and rerun support.

Phase 3 - QA Workflow

- Build scenario review, result dashboard, export, Jira publishing, and manual QA verdict.
- Add defect draft generation and integration with test management tools.

Phase 4 - Intelligence and Scale

- Add RAG over API docs and historical tests, regression impact analysis, and CI/CD integration.
- Improve confidence scoring, flaky test classification, and cross-ticket learning.